



$star = 1;$
 $space = 2 * n - 1;$
 $row = 1;$

5 4 3

3 2 1

$row = 5$
 $space = 5$
 $star = 5$
 $5 < 5$
 $5 < 5$

while (row <= 2 * n + 1) {

$int number = n;$
 $// star$

int i = 1;

while (i <= star) {

→ cout (number);

i++; number--;

// space

int j = 1;

while (j <= space) {

cout (" ");

j++;

}

(number--)

5 4 3 2 1 0 -1

// star

int k = 1;

while (k <= star) {

cout (right number);

k++; right number++;

}

if (row <= n) {

star++;

space = space - 2;

else {

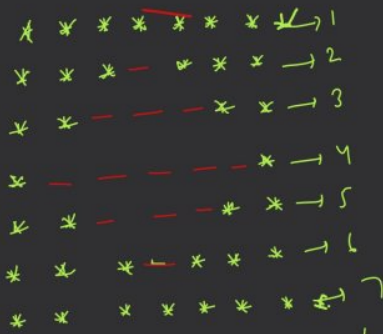
star--;

space = space + 2;

}

$int right number$
 $= number + 1$

5 4 3 2 1 0 1 2
3 4 5



```

int row = 1;
int star = n/2 + 1;
int space = -1;
while (row <= n) {
    // left star

```

```

// space
int j = 1;
while (j <= space) {
    cout << " ";
    j++;
}
// star
int k = 1;
while (k <= star) {
    cout << "* ";
    k++;
}

```

```

if (row == 1 || row == n) {
    // k = 1
}
else {
    // k = 2
}

```

```

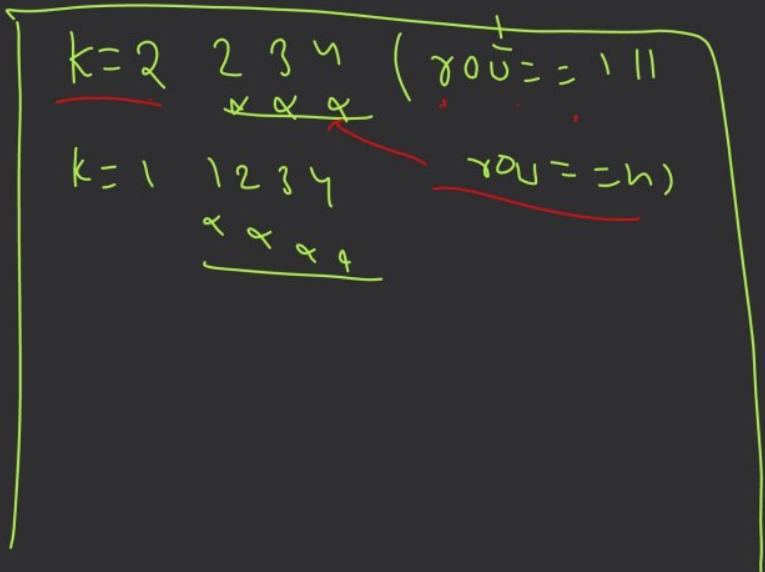
space = space - 2;
star++;

```

1. n=7

2. Count Rows

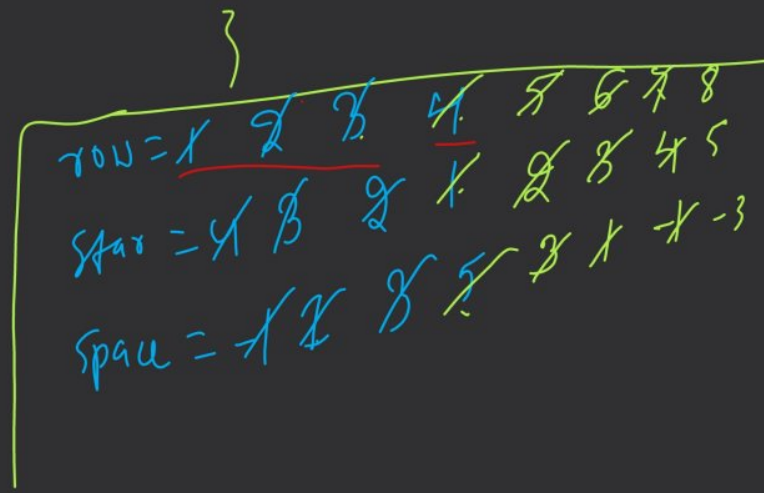
3. Decide the flow of outer loop



```

// Memory
if (row < n/2 + 1) {
    space = space + 1;
    star--;
}

```





$n=5$

Total Min = 6 (row = 1 $\rightarrow$ 6)

while (row $\leq$ $n+1$)
 1, 2, 3, 4, 5, 6

(n=5)



n=5

space = n-1

marks 1 = 80;

marks 2 = 20;

⋮

Array

row = 1

star = 2

space = 3



```

- - - *
- - * * *
- * * * *
* * * * *

```

(n=4)

```

      1
    2 2 2
  3 3 3 3
4 4 4 4 4 4

```

```

int j = 1;
while(j <= start) {
  sort(arral);
  j++;
}

```

spaces = n - 1

row = 1, star = 1

```

      ↓ 1
    2 3 4
  ↓ 3 4 5 6 7
4 5 6 7 8 9 10

```

```

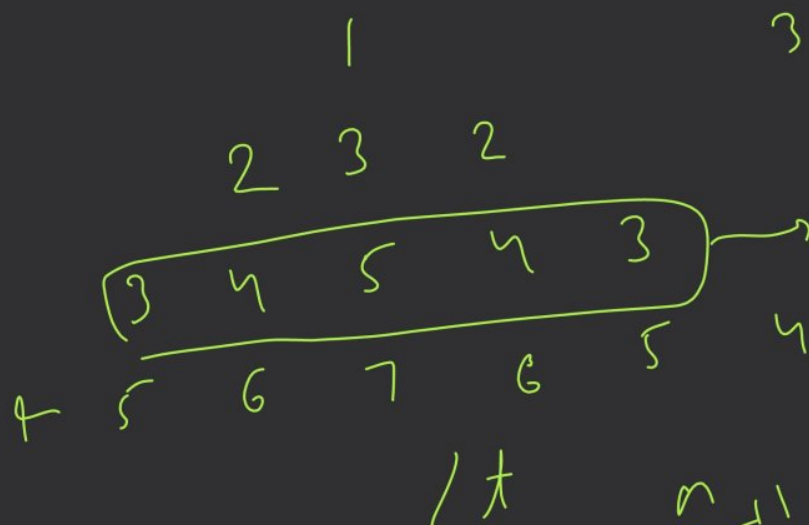
while(row <= n) {
  int val = row;
  int i = 1;
  while(i <= space) {
    sort(arr, val);
    i++;
  }
}

```

```

sort()
row++;
space--;
}

```

Star = 5

$j = \cancel{1} \cancel{2} \cancel{3} \cancel{4} \cancel{5} 6$

$val = \cancel{3} \cancel{4} \cancel{5} \cancel{4} 3$

$\frac{n}{2} - 1$

n

$\begin{pmatrix} 3 \\ 4 \\ 5 \end{pmatrix}$

5 star (Val = 3)

while (j <= star) {

Sort(val);

if (j < $\frac{star}{2} + 1$)

val++;

~~else val--;~~

j++;

}

1 < 3

2 < 3

H.W.

→ Do dry run for missing.

→ 8 Ans. must be done

→ Dry run for boundary cond:-

```

int m1 = 80;
int m2 = 75;
int m3 = 70;
.
.
.
int m10 = 7;

```

data type

{

}

↓

int []

marks

= new

int

[1000]

;

↙

↓

name

↓

creating
memory

↘

Size of
array

(array)

marks = [00 50 0 0 0 +]
 0 1 2 999

marks[0] = 0; marks[2] = 5

X Don't
write
on
editor

```
int [] arr = { 10, 20, 30, 40, 50 };
```

```
- Sort(arr[0]);
```

```
- Sort(arr[1]);
```

```
for (int i=0; i<=4; i++) {
```

```
    i < 5  
    system.out.println(arr[i]);  
}
```

10	20	30	40	50
0	1	2	3	4

